

EduGenie Google Gemini Powered Learning Assistant

Project Description:

EduGenie is a lightweight AI-powered educational assistant designed to simplify and enhance the learning experience through the power of Generative AI. Developed for students, self-learners, and educators. EduGenie provides intelligent educational support by helping users understand concepts, answer questions, generate quizzes, summarize learning materials, and receive personalized learning recommendations. The platform aims to make learning more interactive, accessible, and efficient for users across different academic levels and subject domains.

Built using **FastAPI** for the backend and a responsive **HTML + CSS** frontend, EduGenie combines the capabilities of lightweight local models and cloud-based AI services to deliver fast and accurate educational assistance. Its optimized architecture allows it to run efficiently on resource-constrained devices such as the **Mac M1**, making it suitable for both personal learning and educational projects.

Key features of EduGenie include:

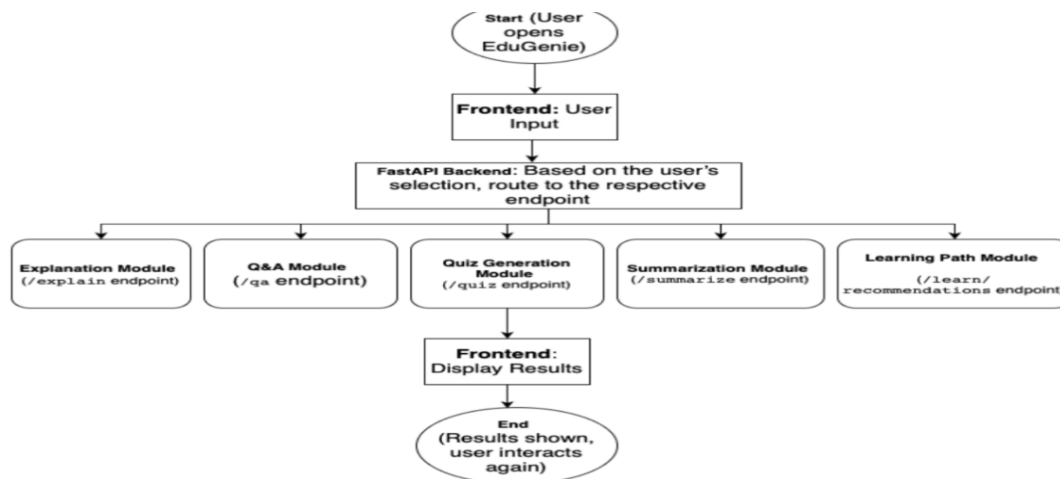
- Intelligent Question Answering
- Simplified Concept Explanations
- AI-Powered Quiz Generation
- Educational Text Summarization
- Personalized Learning Path Recommendations
- Interactive and User-Friendly Interface

Scenario 1: A student wants to learn about oceans and rivers. They ask, "Which is the largest ocean?" EduGenie instantly provides an accurate answer along with additional educational context.

Scenario 2: A student studying the **Pythagoras Theorem** wants to assess their understanding. They select "Generate Quiz," and EduGenie creates topic-specific questions for self-evaluation.

Scenario 3: A learner interested in SQL requests a structured learning roadmap. EduGenie generates a personalized learning path covering beginner, intermediate, and advanced topics with study recommendations and progression guidance.

Technical Architecture:



Description: **EduGenie** is a lightweight AI-powered educational assistant designed to simplify and enhance the learning experience through the power of Generative AI. Developed for students, self-learners, and educators, EduGenie provides intelligent educational support by helping users understand concepts, answer questions, generate quizzes, summarize learning materials, and receive personalized learning recommendations. The platform aims to make learning more interactive, accessible, and efficient for users across different academic levels and subject domains.

Pre-requisites:

Before getting started with the EduGenie Learning Assistant project, ensure you have a basic understanding of Python programming, REST APIs, and fundamental concepts of Artificial Intelligence and Natural Language Processing (NLP). Familiarity with FastAPI, HTML, CSS, and working with APIs will be helpful for developing and integrating different modules of the application. Additionally, install Visual Studio Code (VS Code) or any preferred code editor for efficient coding and project management. Basic knowledge of Git and virtual environments is also recommended for dependency handling and version control.

Python 3.10+

Official Documentation: [Google AI for Developers](#)

Installation Guide: [Tom's Hardware](#)

Popular Tutorial: [Python Programming Tutorial - Full Course for Beginners](#)

Installation Steps:

Download the latest Python 3.10+ installer for your operating system from the official website.

Run the installer and ensure you check the box that says "Add Python to PATH".

Follow the installation prompts to complete the setup.

Verify the installation by opening a terminal or command prompt and typing `python --version`.

FastAPI Framework –

Official Documentation: [FastAPI](#)

User Guide: [FastAPI](#)

Popular Tutorial: [FastAPI Crash Course](#)

HTML & CSS – Basic templating used in `/templates` and `/static`

Google Gemini API Key

Official Documentation: [Google AI for Developers](#)

Setup Guide: [GeeksforGeeks](#)

Popular Tutorial: [How to Use Google Gemini API Key](#)

Setup Steps:

Visit the Google AI Studio and sign in with your Google account.

Create a new project and enable the Gemini API.

Generate an API key and securely store it.

Uvicorn (ASGI Server)

Official Documentation: [PyPI](#)

Installation Steps:

Install Uvicorn using pip: `pip install uvicorn`

Jinja2 (HTML Templating Engine)

Official Documentation: [Jinja2 Documentation](#)

Installation Steps:

Install Jinja2 using pip: `pip install jinja2`

Project Workflow:

Epic 1. Model Selection and Architecture

Story 1: Configure Gemini 1.5 Pro and LaMini-Flan-T5 models for question answering, summarization, quiz generation, learning recommendations, and concept explanation.

Epic 2. Core Functionalities Development

Story 1: Implement Explanation, QnA, Quiz, Summary, and Learning Path modules with AI-powered educational functionalities.

Story 2: Create RESTful API endpoints and connect each endpoint with the respective module logic.

Epic 3. Frontend Development

Story 1: Develop a responsive frontend interface with task dropdowns, text input areas, submit buttons, and CSS styling.

Story 2: Integrate frontend forms with FastAPI backend using POST requests and display real-time outputs.

Epic 4. Deployment

Story 1: Deploy and run the EduGenie application locally using Uvicorn and FastAPI.

Story 2: Test all functionalities, including question answering, explanation generation, quiz generation, summarization, and personalized learning recommendations.

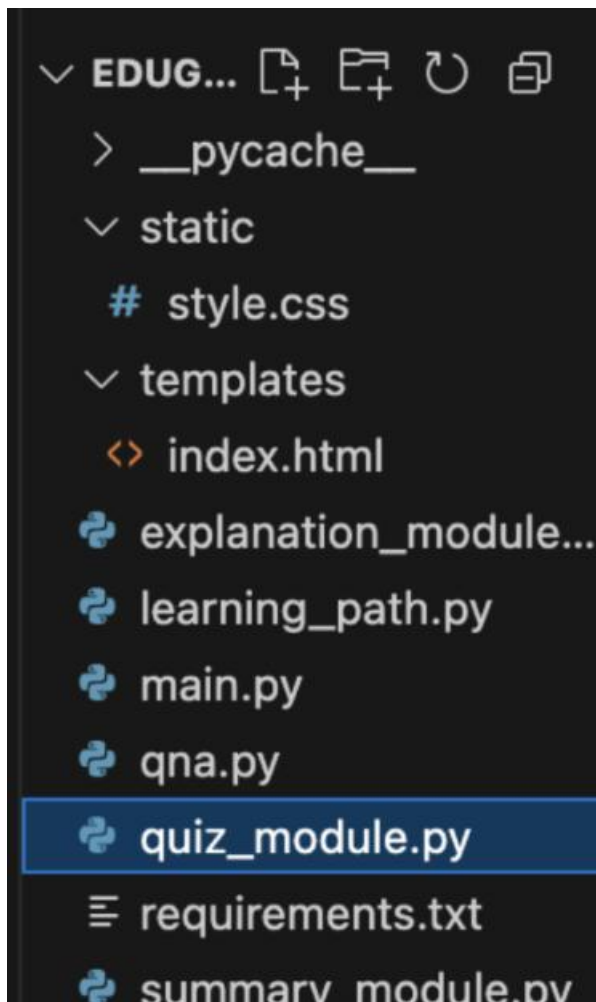
EPIC 1: Model Selection and Architecture

It focused on selecting suitable AI models and designing the foundational architecture of the EduGenie Learning Assistant. The objective of this activity was to ensure efficient educational assistance, modular development, and smooth integration between frontend, backend, and AI-powered functionalities.

For advanced educational functionalities such as Question Answering (Q&A), Summarization, Quiz Generation, and Personalized Learning Recommendations, **Google Gemini 1.5 Pro** was selected through API integration. Gemini 1.5 Pro was chosen because of its advanced reasoning capabilities, contextual understanding, structured response generation, and cloud inference support. The model enables EduGenie to generate intelligent and human-like educational content while maintaining high accuracy and efficiency across multiple learning tasks.

For concept explanation functionality, **LaMini-Flan-T5-783M** was selected as the local lightweight AI model. This model is instruction-tuned and specifically suitable for generating simplified explanations for complex educational topics. Since the model is CPU-compatible and lightweight, it improves local execution efficiency and reduces dependency on cloud resources for simpler tasks. The model helps learners understand difficult concepts in a concise, beginner-friendly, and highly readable manner.

Along with model selection, a modular folder architecture was designed to organize the project efficiently. The architecture separated backend modules, frontend resources, AI functionalities, and dependency management into independent files for better scalability, maintainability, and debugging.



The folder structure included dedicated Python modules such as:

[qna.py](#) for Question Answering

explanation_module.py for concept explanation

quiz_module.py for quiz generation

summary_module.py for summarization

learning_path.py for personalized learning recommendations

The [main.py](#) file acted as the central FastAPI application responsible for routing and backend integration. Frontend resources were managed through the **templates** and **static** folders containing **index.html** and **style.css**, respectively. Additionally, the requirements.txt file stored all the required Python dependencies needed to run the EduGenie application successfully.

EPIC 2: Core Functionalities Development

Explanation Module: EduGenie utilizes the LaMini-Flan-T5 model, a lightweight yet powerful generative AI, to deliver educational content in a simplified and highly readable manner. This model is specifically fine-tuned to provide concise, context-aware responses that break down complex topics into easily understandable language. By focusing on clarity and brevity, EduGenie ensures that learners with minimal background knowledge or technical experience can grasp essential concepts

without feeling overwhelmed. This makes it particularly valuable for beginners, school students, or self-learners seeking straightforward explanations. The integration of LaMini-Flan-T5 empowers EduGenie to act as a reliable and accessible study companion for foundational learning across subjects.

```
lanation_module.py
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

# Load the improved model for explanations
explain_tokenizer = AutoTokenizer.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")
explain_model = AutoModelForSeq2SeqLM.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")

def explain_topic(topic: str) -> str:
    input_text = f"Explain the concept of '{topic}' in a simple and clear way for a school student."
    inputs = explain_tokenizer(input_text, return_tensors="pt")

    outputs = explain_model.generate(
        **inputs,
        max_new_tokens=150,
        temperature=0.7,
        top_k=50,
        top_p=0.95,
        do_sample=True
    )

    explanation = explain_tokenizer.decode(outputs[0], skip_special_tokens=True)
    return explanation
```

QnA Module: EduGenie is powered by Gemini 1.5 Pro, enabling it to handle a wide range of general knowledge and academic question-answering tasks with precision. Its advanced understanding and contextual capabilities make it an effective tool for learners seeking accurate, AI-driven assistance across diverse educational topics and subjects.

```
def answer_question_with_gemini(question: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        response = model.generate_content(question)
        return response.text.strip()
    except Exception as e:
        return f"⚠ Error in QnA: {e}"
```

Quiz Module: EduGenie generates three multiple-choice questions (MCQs) from a given passage, each containing four carefully crafted options. It utilizes the Gemini model to comprehend the context and meaning of the passage, ensuring that the questions are relevant and the distractors are plausible. The final output is structured in JSON format for easy integration.

It sends a structured prompt to the model, expecting a valid JSON response with each question containing four options and a correct answer. To ensure proper parsing, it cleans any Markdown code blocks using the `clean_json_blockfunction`. The cleaned response is then parsed into a Python list. If an error occurs during generation or parsing, it returns a detailed error message for debugging.

```
def clean_json_block(text):
    # Remove Markdown ```json code fences
    return re.sub(r"```(?:json)?\n(.*?)```", r"\1", text, flags=re.DOTALL).strip()

def generate_quiz(text: str) -> list:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"""
You are a quiz generator.

From the following passage, create 3 multiple-choice questions. Each question should include:
- A "question"
- A list of 4 "options"
- A correct "answer" that must exactly match one of the options.

Format your output as **valid JSON**, like this:
[
  {{
    "question": "What is ...?",
    "options": ["A", "B", "C", "D"],
    "answer": "A"
  }}
]

Passage:
{text}
"""
        response = model.generate_content(prompt)
        quiz_text = response.text.strip()

        # 🧹 Clean markdown code blocks if any
        cleaned_text = clean_json_block(quiz_text)
```

Summary Module: This feature leverages Gemini’s generative capabilities to summarize long paragraphs into concise, easy-to-understand versions, making it ideal for quick revision. It ensures the core information is retained while eliminating redundancy, helping learners absorb key points efficiently without losing clarity or context.

Learning Path Module: The `get_learning_recommendations` function uses Google Gemini to generate a personalized, structured learning path for any given topic. Crafting a detailed prompt, it instructs the AI to suggest beginner to advanced concepts, organized by difficulty, and supported with useful resources such as videos, articles, or books. It ensures adaptability to the learner’s level. The function safely handles API responses and errors, providing either the generated text or a helpful error message if the response is invalid or incomplete.

```
def get_learning_recommendations(topic):
    prompt = f"""
    You are an AI tutor. The student wants to learn about: {topic}.
    Suggest a structured and adaptive learning path including key topics, order of learning, and resources (
    Include beginner, intermediate, and advanced levels if needed.
    """
    try:
        response = model.generate_content(prompt)
        print("🗨️ Gemini raw response:", response)

        if hasattr(response, "text"):
            return response.text
        elif hasattr(response, "parts") and response.parts:
            return response.parts[0].text
        else:
            return "❌ Could not extract content from Gemini response."
    except Exception as e:
        import traceback
        traceback.print_exc() # ✅ This gives full error trace
        return f"❌ Error occurred: {str(e)}"
```

It focused on developing the backend infrastructure of the EduGenie Learning Assistant using the FastAPI framework. In this activity, RESTful API endpoints were created for each educational functionality so that the frontend interface could communicate efficiently with the AI-powered backend modules. FastAPI was selected because of its high performance, asynchronous request handling, simplicity, and easy API integration capabilities. The backend architecture was designed in a modular manner, where each endpoint was connected to its respective AI functionality. Separate API routes were developed for Question Answering, Concept Explanation, Quiz Generation, Summarization, and Personalized Learning Recommendations. These endpoints handled user requests, processed input data, generated AI-powered outputs, and returned structured JSON responses to the frontend interface.

The following RESTful API endpoints were implemented:

- /qa → Handles educational question answering
- /explain → Generates simplified explanations for topics
- /quiz → Creates AI-generated multiple-choice quizzes
- /summarize → Summarizes lengthy educational content
- /learn/recommendations → Generates personalized learning paths

This image shows the implementation of:

Q&A GET API

Explanation POST API

Summarization POST API

This second image shows the implementation of:

Quiz Generation POST API

Learning Recommendation GET API


```
66
67 # Quiz Generation - POST API
68 @app.post("/quiz")
69 async def quiz_api(request: Request):
70     data = await request.json()
71     text = data.get("text")
72     if not text:
73         return JSONResponse(content={"error": "Please provide text for quiz."}, status_code=400)
74     quiz = generate_quiz(text)
75     print("Generated quiz:", quiz) # [X] DEBUG
76     return JSONResponse(content={"quiz": quiz})
77
78 # Learning Recommendations - GET API
79 @app.get("/learn/recommendations")
80 async def learning_recommendation_api(topic: str = Query(...)):
81     recommendation = get_learning_recommendations(topic)
82     return {"topic": topic, "recommendation": recommendation}
83
```

Each endpoint receives user input through request parameters or JSON data, validates the input, calls the corresponding module logic, and finally returns the generated educational response in JSON format. Error handling was also implemented to ensure proper validation when required input fields are missing.

This backend API integration established smooth communication between the frontend interface and AI modules, enabling real-time educational assistance and efficient response generation within the EduGenie Learning Assistant.

EPIC 3: Frontend Development

It focused on developing the frontend interface of the EduGenie Learning Assistant using HTML, CSS, and Jinja2 templates. The main objective of this activity was to create a simple, responsive, and user-friendly interface through which users could easily interact with different AI-powered educational functionalities.

The frontend interface was developed inside the **index.html** file, where separate forms were created for functionalities such as:

- Question Answering (QnA)

- Concept Explanation

- Quiz Generation

- Text Summarization

- Personalized Learning Recommendations

Each section included:

- Input fields for user queries

- Task-specific placeholders

- Submit buttons for sending requests

- Output containers for displaying generated results

The frontend styling was managed through the **style.css** file, which provided:

- Responsive web design

- Styled buttons and text areas

```

templates > <> index.html > ...
 2  <html lang="en">
 3  <head>
86  </head>
87  <body>
88      <h1>Welcome to EduGenie 🤖!</h1>
89      <p>Your personal AI tutor for learning support!</p>
90
91      <!-- Q&A -->
92      <form id="qaForm">
93          <label><strong>Ask EduGenie a Question:</strong></label><br><br>
94          <input type="text" id="question" placeholder="Why is the sky blue?" required>
95          <button type="submit">Get Answer</button>
96      </form>
97      <div id="qaResult" class="output"></div>
98
99      <!-- Explanation -->
100     <form id="explainForm">
101         <label><strong>Need an Explanation?</strong></label><br><br>
102         <input type="text" id="topic" placeholder="Photosynthesis" required>
103         <button type="submit">Explain</button>
104     </form>
105     <div id="explanationResult" class="output"></div>
106
107     <!-- Summary -->
108     <form id="summaryForm">
109         <label><strong>Summarize a Paragraph:</strong></label><br><br>
110         <input type="text" id="summaryText" placeholder="Paste long content to summarize" required>
111         <button type="submit">Summarize</button>
112     </form>
113     <div id="summaryResult" class="output"></div>
114
115     <!-- Quiz -->
116     <form id="quizForm">
117         <label><strong>Generate a Quiz:</strong></label><br><br>
118         <input type="text" id="quizText" placeholder="Solar System" required>
119         <button type="submit">Generate Quiz</button>
120     </form>

```

Organized result containers

Improved user experience and readability

The interface was designed to ensure smooth user interaction while maintaining a clean and visually structured layout for educational assistance.

This activity successfully established the user interaction layer of EduGenie, allowing learners to access different educational services through a single integrated web interface.

It focused on integrating the frontend interface with the FastAPI backend to enable real-time communication between users and the AI-powered educational modules. In this activity, form submissions from the frontend were connected to backend API endpoints using POST requests.

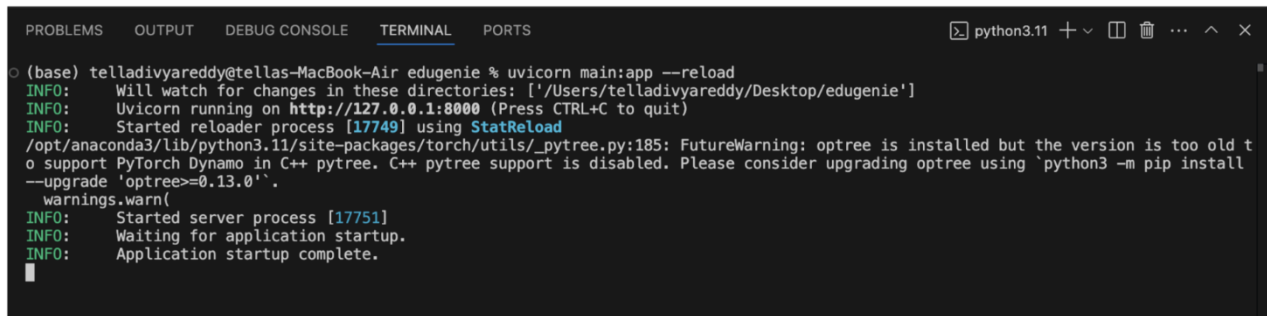
When a user entered a query or selected a specific educational task, such as QnA, Explanation, Quiz Generation, Summarization, or Learning Recommendations, the frontend captured the input data and sent it to the corresponding FastAPI API endpoint. The backend then processed the request using the appropriate AI module and generated the required educational response.

The generated output was dynamically displayed below the respective input section in real-time without requiring manual page refreshes. This integration ensured smooth interaction,

faster response delivery, and an interactive learning experience for users. It also established seamless communication between the HTML frontend and FastAPI backend architecture of the EduGenie Learning Assistant.

EPIC 4: Deployment

- It focused on deploying and running the EduGenie Learning Assistant in a local development environment using FastAPI and Uvicorn. This activity ensured that the complete frontend-backend system was functioning correctly and could be accessed through a web browser for real-time interaction.
- The application was executed locally using the following command:
 - `uvicorn main:app --reload`
- The **--reload** flag enabled automatic server reloading whenever changes were made to the project files, making development and testing more efficient.
- After successful execution, the application became accessible through the local server URL:
 - <http://127.0.0.1:8000>



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
python3.11 + - [] ... ^ x

(base) telladivyaireddy@tellas-MacBook-Air edugenie % uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['/Users/telladivyaireddy/Desktop/edugenie']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [17749] using StatReload
/opt/anaconda3/lib/python3.11/site-packages/torch/utils/_pytree.py:185: FutureWarning: optree is installed but the version is too old to support PyTorch Dynamo in C++ pytree. C++ pytree support is disabled. Please consider upgrading optree using `python3 -m pip install --upgrade 'optree>=0.13.0'`.
warnings.warn(
INFO: Started server process [17751]
INFO: Waiting for application startup.
INFO: Application startup complete.
  
```

Fig. Showing the Application can be accessed

- The image shown above illustrates the terminal output after running the application successfully. It displays:
 - Uvicorn server initialization
 - Local server address
 - Application startup completion

- Live backend execution status
- This confirmed that the FastAPI backend was running properly and ready to process educational requests from the frontend interface.
- This activity successfully established the local deployment environment for EduGenie and enabled real-time testing of all AI-powered educational functionalities.

Functional testing:

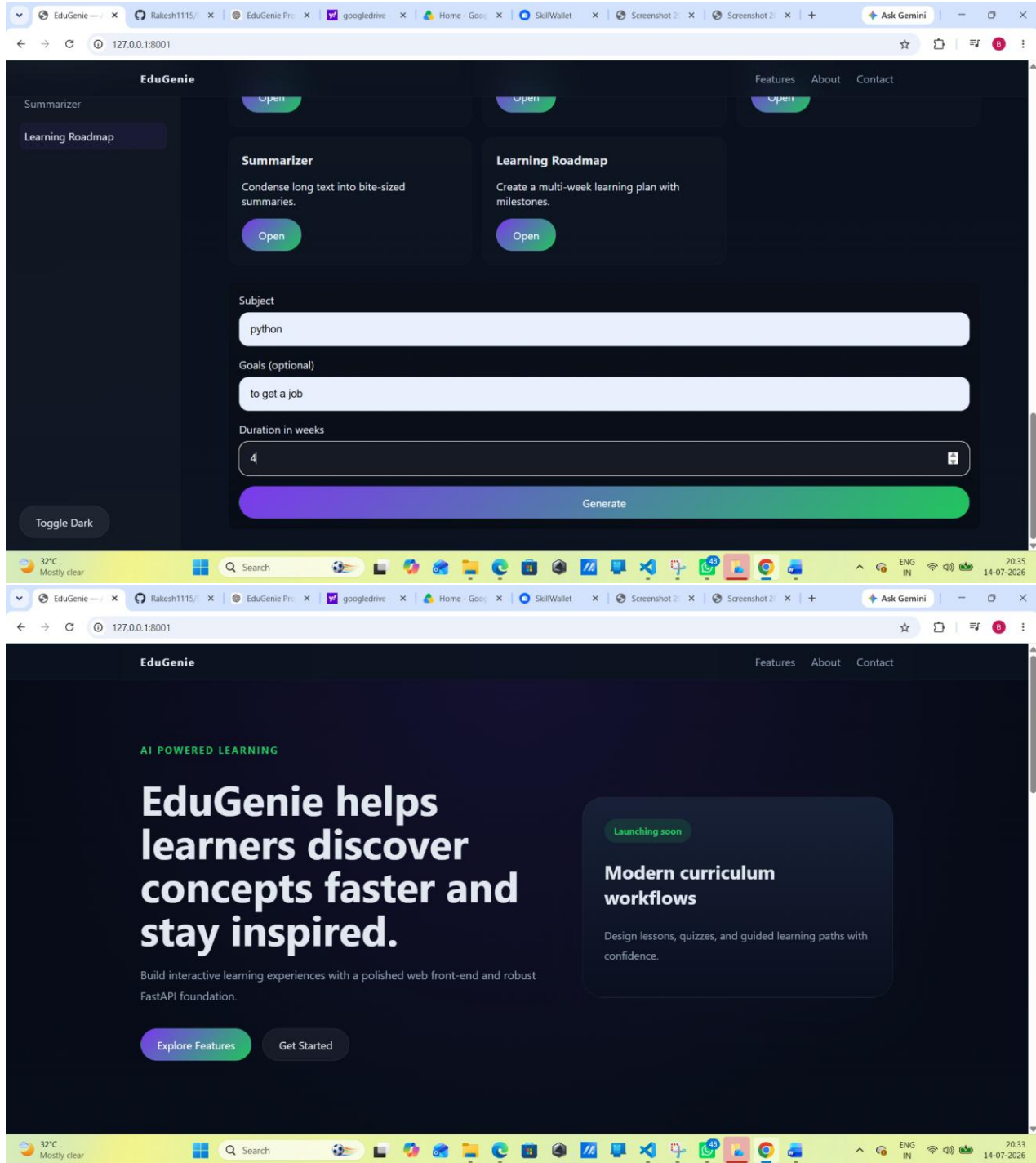
Ask a question

Get an explanation

Generate a quiz

Summarize content

Get a personalized learning plan



The screenshot displays the EduGenie web application interface, which is designed for AI-powered learning. The interface is dark-themed and features a sidebar on the left with navigation options: "Summarizer" and "Learning Roadmap". The main content area is divided into two columns, each with a corresponding "Open" button. The "Summarizer" column includes a description: "Condense long text into bite-sized summaries." The "Learning Roadmap" column includes a description: "Create a multi-week learning plan with milestones." Below these columns, there are input fields for "Subject" (containing "python"), "Goals (optional)" (containing "to get a job"), and "Duration in weeks" (containing "4"). A large "Generate" button is positioned at the bottom of the input fields. The top of the interface shows a navigation bar with "Features", "About", and "Contact" links. The bottom of the interface shows a Windows taskbar with various application icons and a system tray displaying the date and time (20:33, 14-07-2026).

EduGenie Features About Contact

Summarizer Learning Roadmap

Summarizer
Condense long text into bite-sized summaries.
Open

Learning Roadmap
Create a multi-week learning plan with milestones.
Open

Subject
python

Goals (optional)
to get a job

Duration in weeks
4

Generate

Toggle Dark

EduGenie Features About Contact

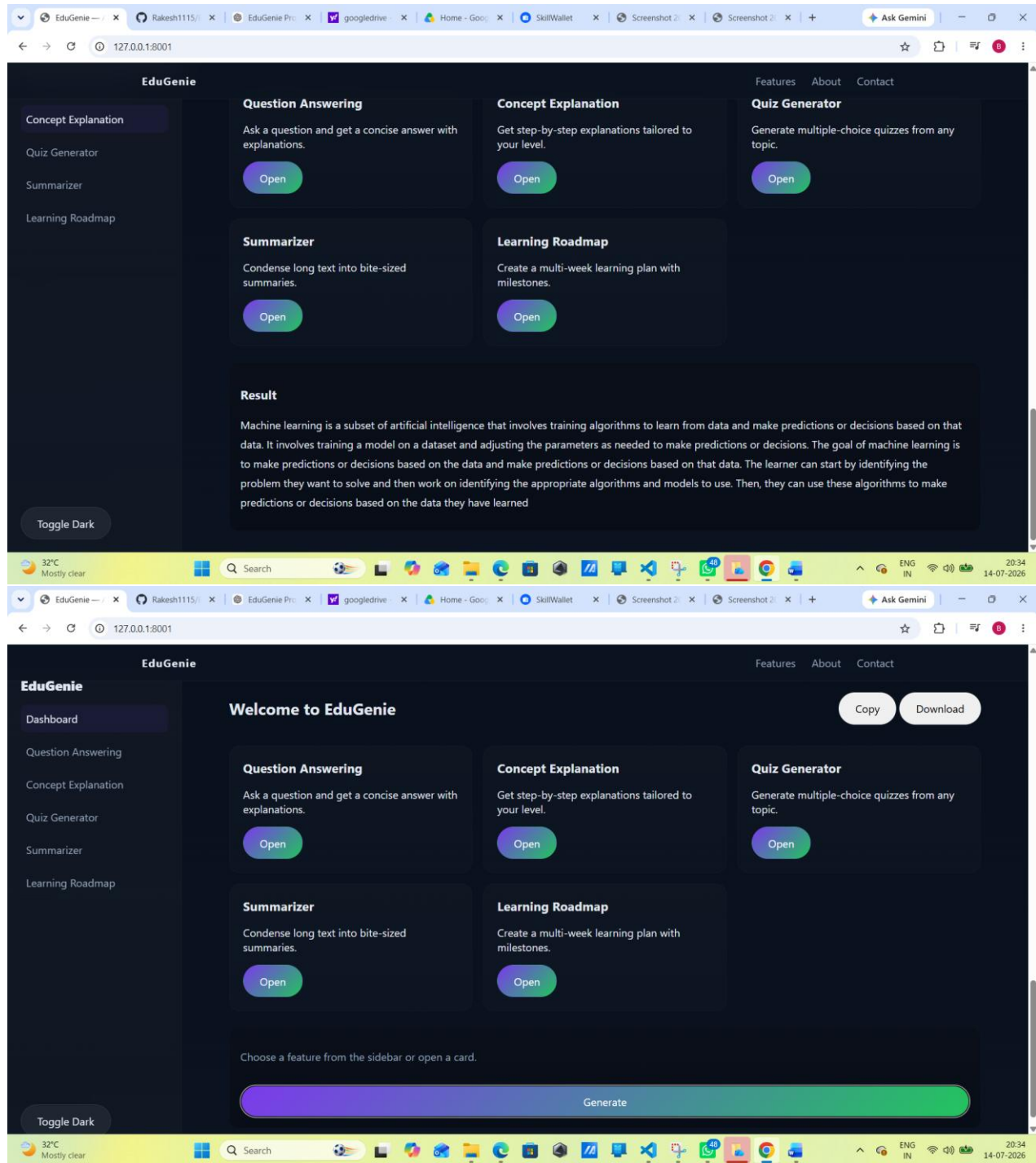
AI POWERED LEARNING

EduGenie helps learners discover concepts faster and stay inspired.

Build interactive learning experiences with a polished web front-end and robust FastAPI foundation.

Explore Features Get Started

Modern curriculum workflows
Design lessons, quizzes, and guided learning paths with confidence.



The screenshot displays the EduGenie web application interface. The browser's address bar shows the URL 127.0.0.1:8001. The application has a dark theme and a sidebar on the left with navigation links: Concept Explanation, Quiz Generator, Summarizer, and Learning Roadmap. The main content area features five interactive cards, each with a description and an 'Open' button:

- Question Answering:** Ask a question and get a concise answer with explanations.
- Concept Explanation:** Get step-by-step explanations tailored to your level.
- Quiz Generator:** Generate multiple-choice quizzes from any topic.
- Summarizer:** Condense long text into bite-sized summaries.
- Learning Roadmap:** Create a multi-week learning plan with milestones.

Below these cards is a 'Result' section containing a paragraph about machine learning. At the bottom of the main content area is a 'Generate' button. The top right corner of the application includes links for 'Features', 'About', and 'Contact'. The browser's taskbar at the bottom shows the system clock as 20:34 on 14-07-2026, with a temperature of 32°C and 'Mostly clear' weather.

Conclusion:

The EduGenie Learning Assistant successfully demonstrates the use of Generative AI in education by providing intelligent and personalized learning support. Using Google Gemini 1.5 Pro and LaMini-Flan-T5, the system efficiently performs Question Answering, Concept Explanation, Quiz Generation, Summarization, and Personalized Learning Recommendations.

The project was developed using FastAPI, HTML, CSS, and Jinja2 with a modular architecture that ensured smooth frontend-backend integration, scalability, and maintainability. Through successful implementation, testing, and local deployment, EduGenie proved to be an effective AI-powered educational platform that enhances learning experiences and provides real-time academic assistance for students and self-learners.